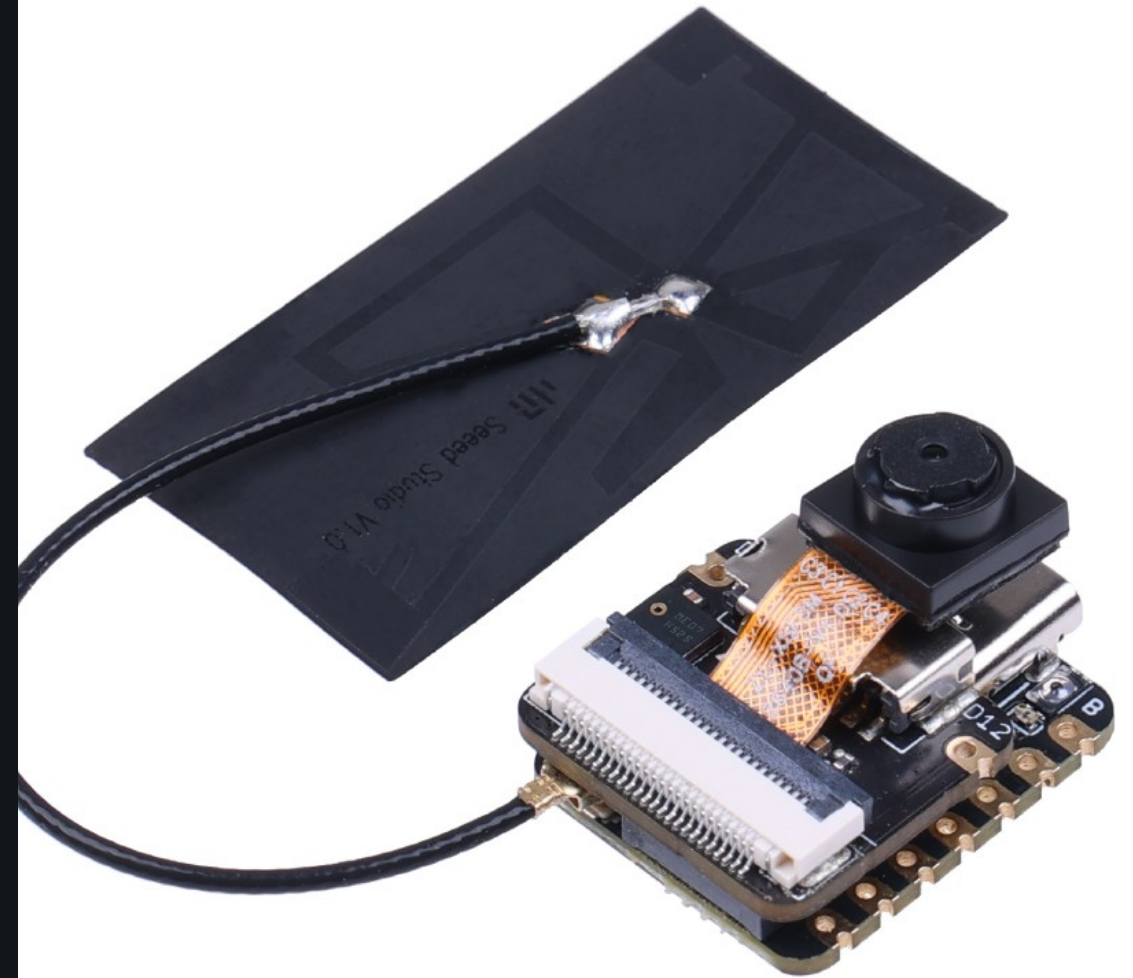


Programming responsive hardware

Microcontrollers, GPIO, and nonblocking timing



Microcontrollers



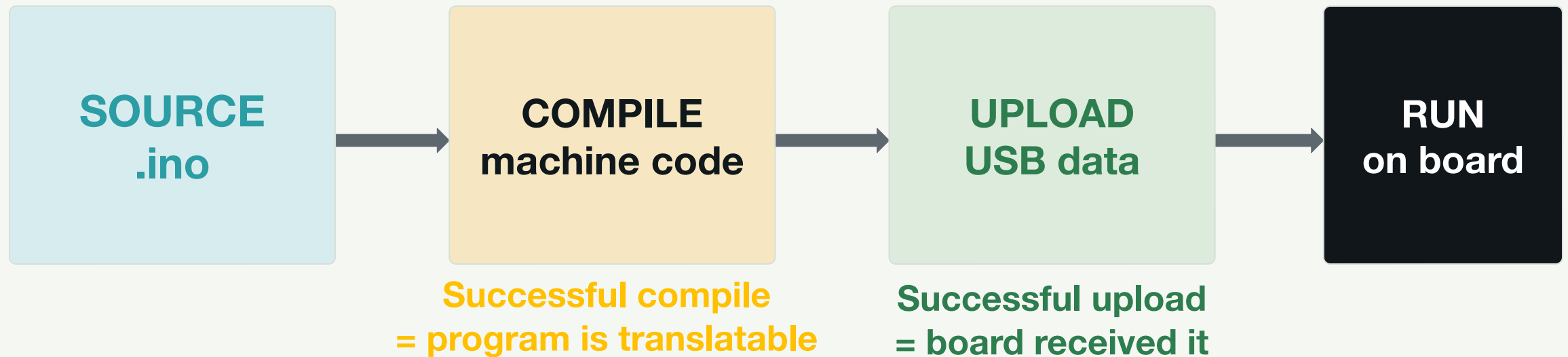
— processor executes instructions

— memory holds the uploaded program

— pins connect computation to hardware

Unlike a laptop, the board is built to live inside another device.

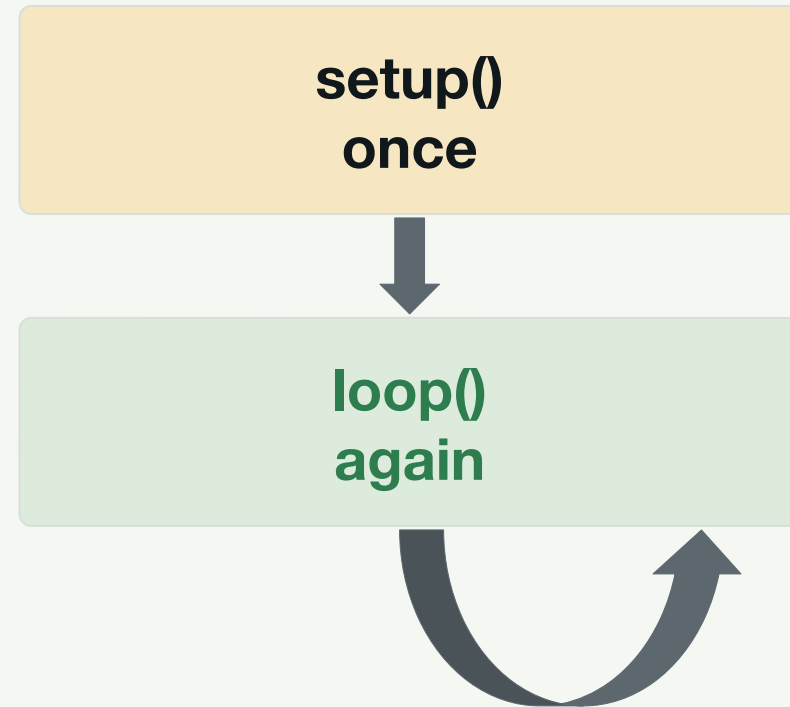
Compiling and Uploading



Neither proves the code has the correct behavior

setup() and loop()

```
void setup() {  
  Serial.begin(115200);  
}  
  
void loop() {  
  Serial.println("again");  
}
```



Serial output as an observation channel

```
void setup() {  
  Serial.begin(115200);  
}  
  
void loop() {  
  if (Serial) {  
    Serial.print("t_ms=");  
    Serial.println(millis());  
  }  
}
```

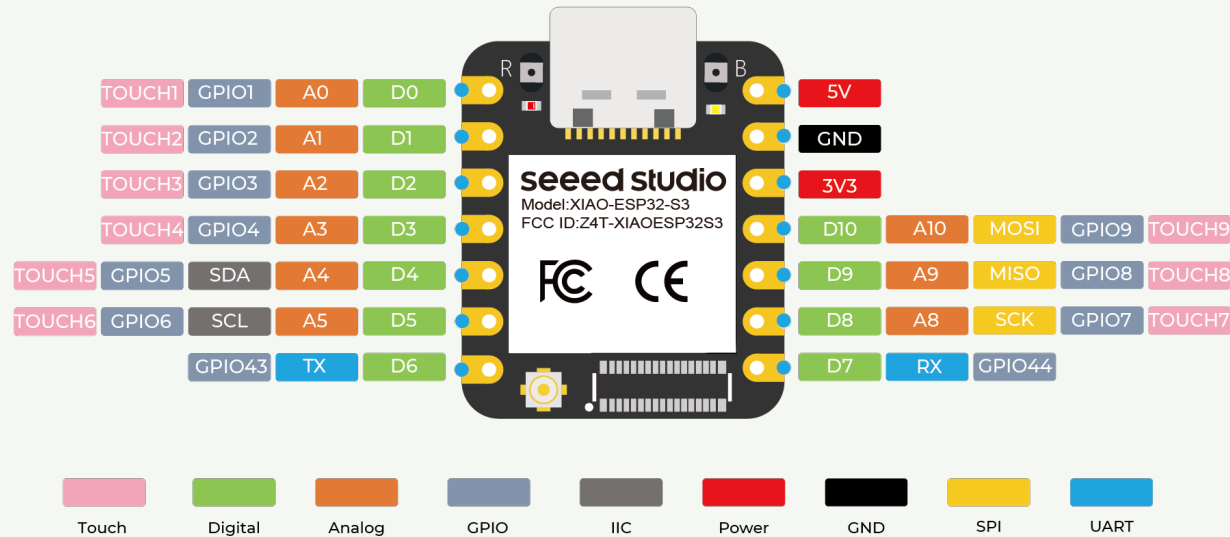
Use 115200 in the sketch and monitor.
Keep the program able to run with the monitor closed.

Serial Monitor

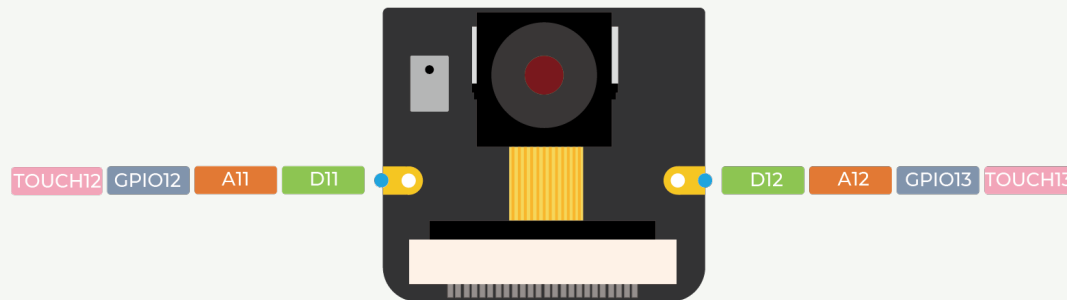
```
t_ms=100  
t_ms=100  
t_ms=101  
t_ms=101
```

Illustrative output. A fast loop can print the same millisecond twice.

General-Purpose Input/Output (GPIO)



```
const int LED_PIN = 3;  
// GPIO3 = board D2  
pinMode(LED_PIN,  
        OUTPUT);
```

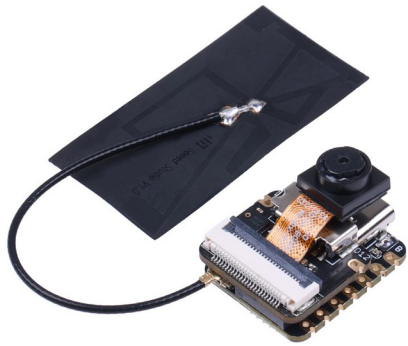


software name (LED_PIN)

board label (D2)

electrical pad

One LED, one complete circuit



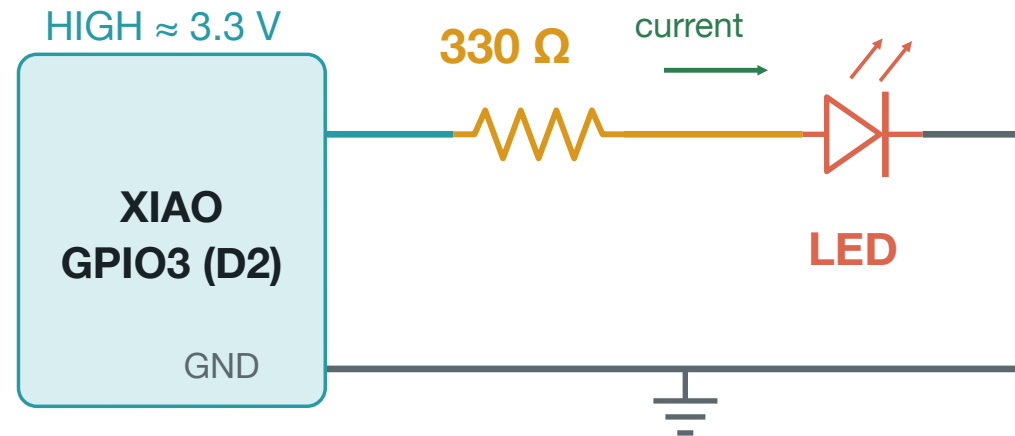
XIAO ESP32-S3 Sense



LED



330 Ω resistor



GPIO3 is the pad labeled D2.
The resistor and LED are in series.

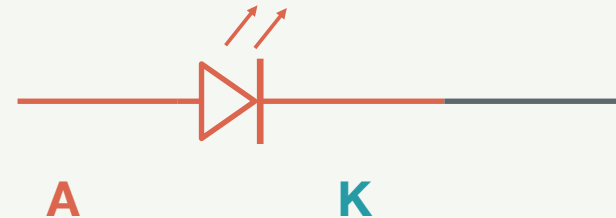
LED polarity: package and symbol



Before inserting the LED

Anode (A): usually the longer lead

Cathode (K): shorter lead and flat edge



The bar marks the cathode.

The small arrows represent emitted light.

The resistor sets the LED current



orange · orange · brown · gold

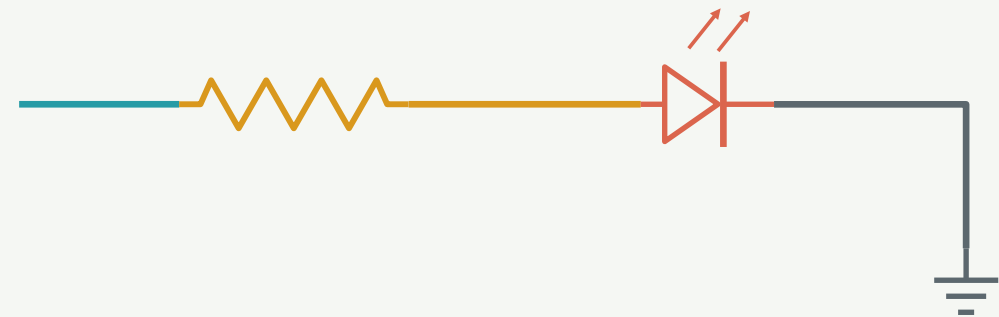
$$33 \times 10 \, \Omega = 330 \, \Omega$$

Gold band: $\pm 5\%$ tolerance

Example values, not a measured LED specification.
A larger series resistance gives less current.

GPIO HIGH
 $\approx 3.3 \, \text{V}$

Across resistor
 $3.3 - 2.0 = 1.3 \, \text{V}$



Red LED example: $V_F \approx 2.0 \, \text{V}$

$$I \approx (3.3 - 2.0) / 330$$
$$\approx 0.0039 \, \text{A} = 3.9 \, \text{mA}$$

Configuring and Driving an Output

```
const int LED_PIN = 3; // D2

void setup() {
  pinMode(LED_PIN, OUTPUT);
}

void loop() {
  digitalWrite(LED_PIN, HIGH);
}
```



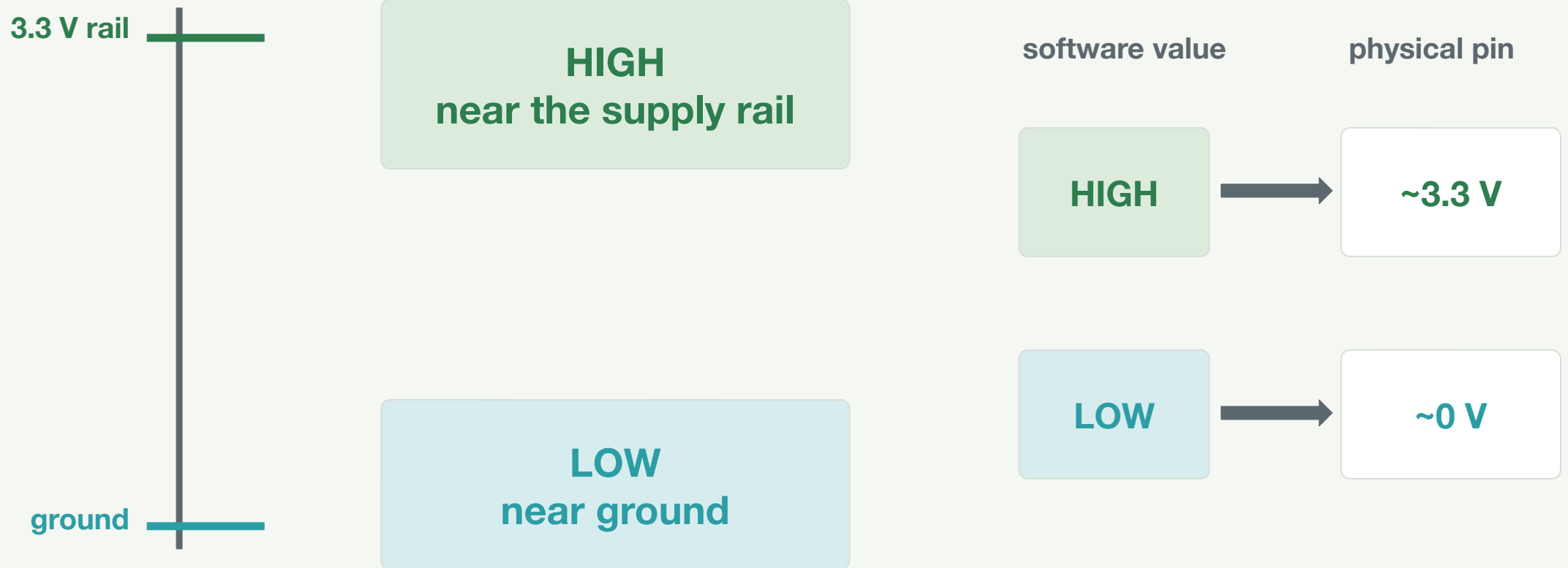
OUTPUT
the pin drives the circuit



HIGH
the output moves toward 3.3 V

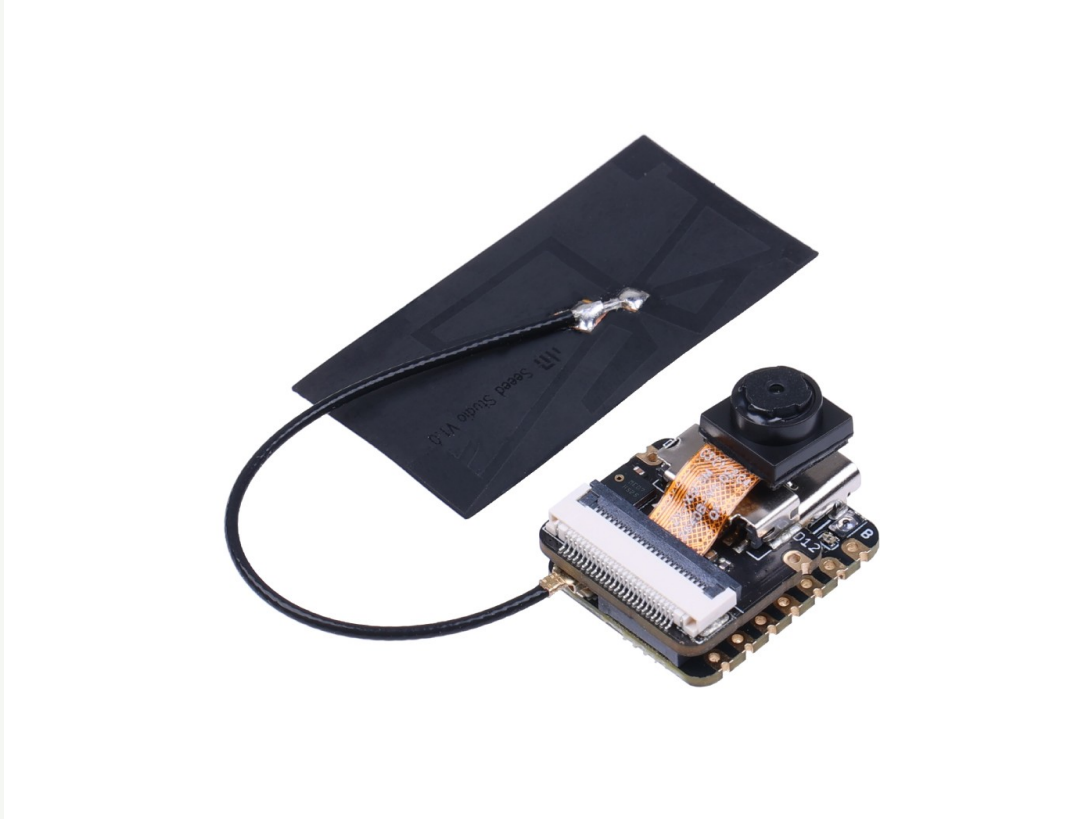
Correct pin + correct role + correct level + correct circuit

GPIO Voltage Levels



L02 will examine the voltage ranges that digital inputs interpret as 0 and 1.

Active-Low Outputs



`digitalWrite(LED_BUILTIN, LOW)`



LED ON

HIGH -> LED OFF

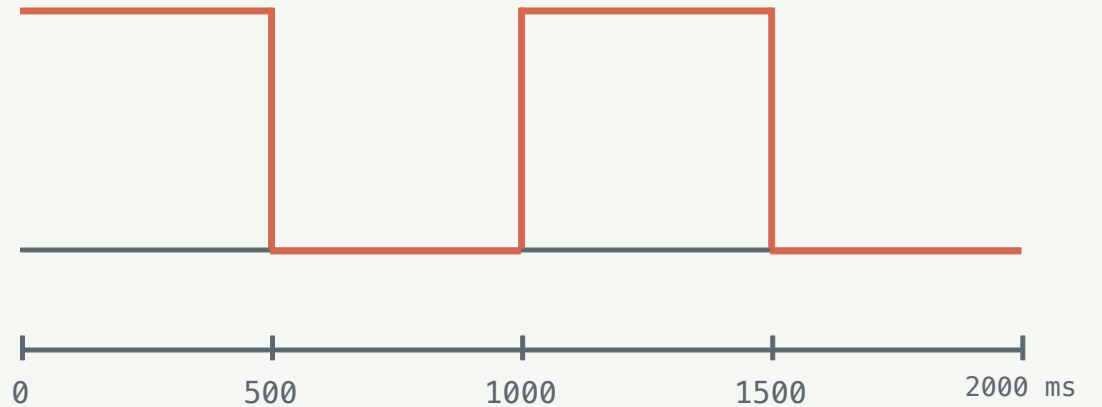
The LED is connected from 3.3 V to the pin: LOW sinks current and turns it on.

Timing

Blocking Timing

```
digitalWrite(RED_LED, HIGH);  
delay(500);  
digitalWrite(RED_LED, LOW);  
delay(500);
```

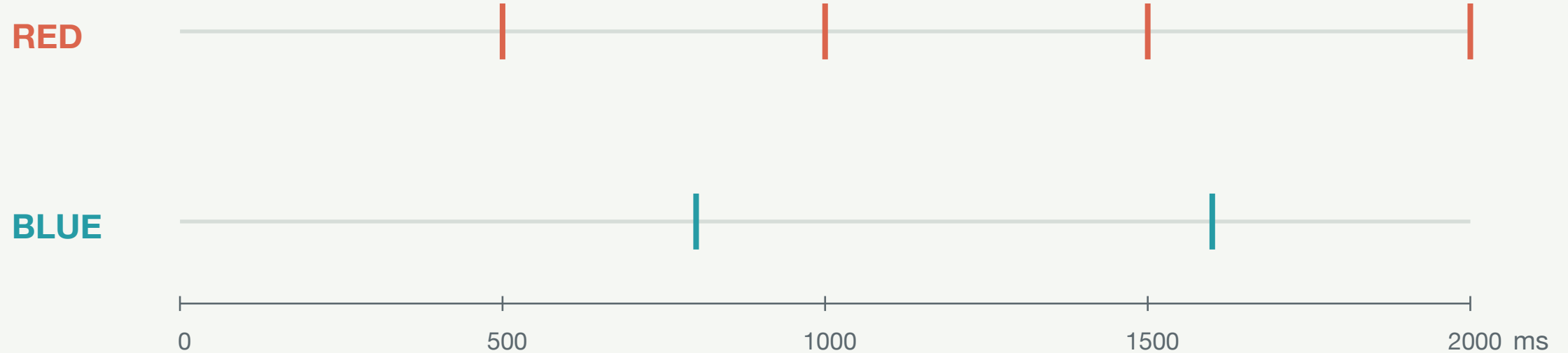
RED_LED



The LED holds its level while the main loop waits.

Two outputs, two timing intervals

Red toggles every 500 ms. Blue toggles every 800 ms.



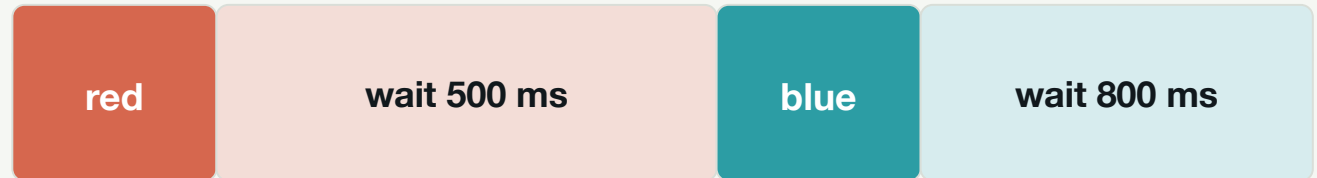
Both outputs start LOW at $t = 0$. Each mark is a toggle.

How can one loop keep both schedules?

Blocking Delays

```
// Pseudocode: toggle one  
output  
toggleRed();  
delay(500);  
  
toggleBlue();  
delay(800);
```

actual loop timeline



next RED change: 1300 ms later

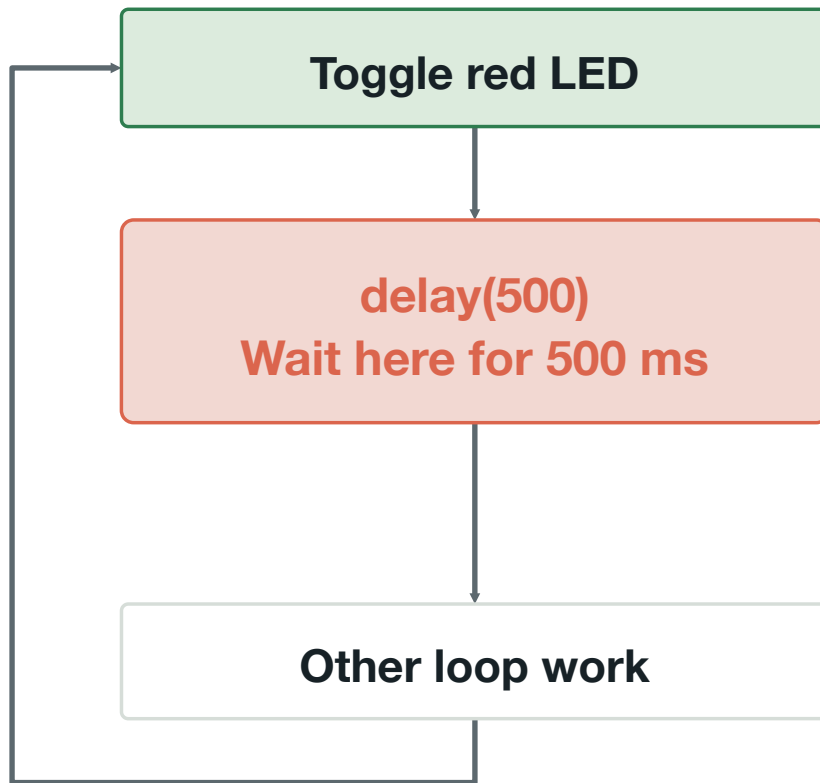
**Later work in this loop waits
while delay() is running.**

This is blocking timing.

Blocking and nonblocking timing

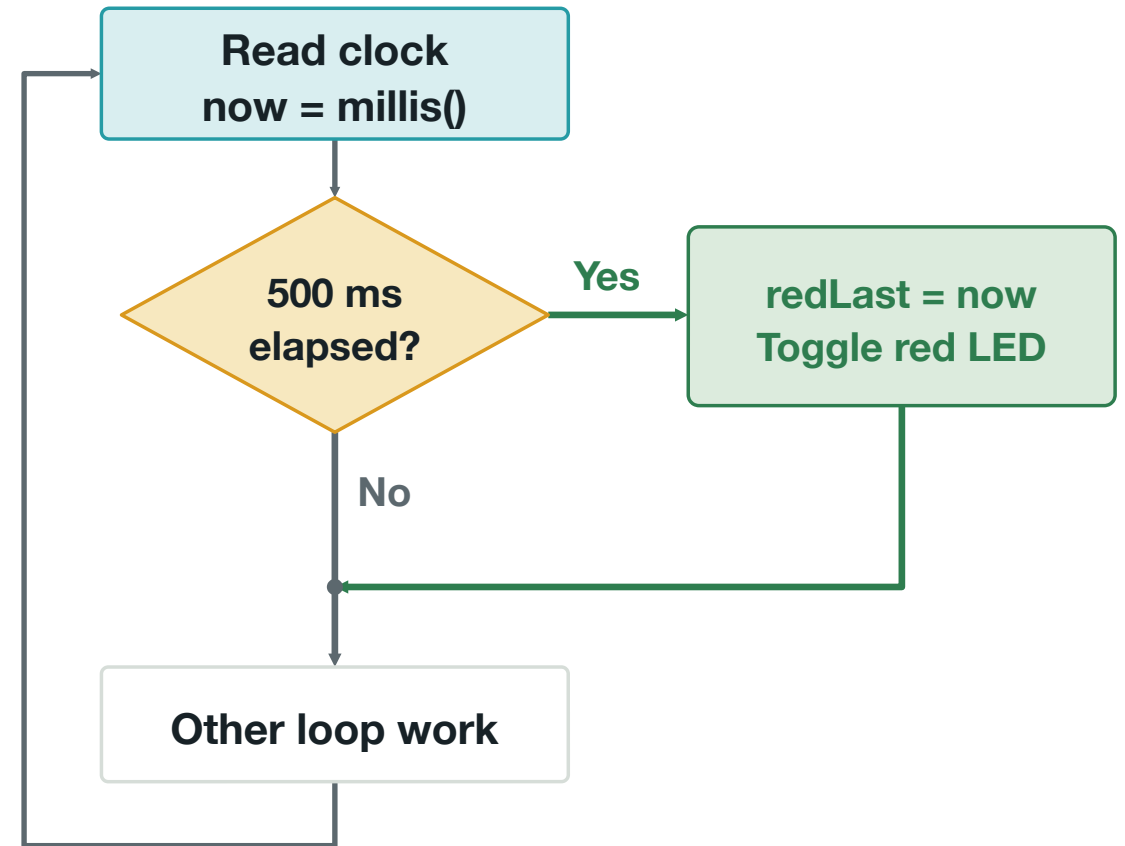
Goal: toggle the red LED about every 500 ms while other work continues.

Blocking with delay()



Later loop work waits too.

Nonblocking clock check



next pass

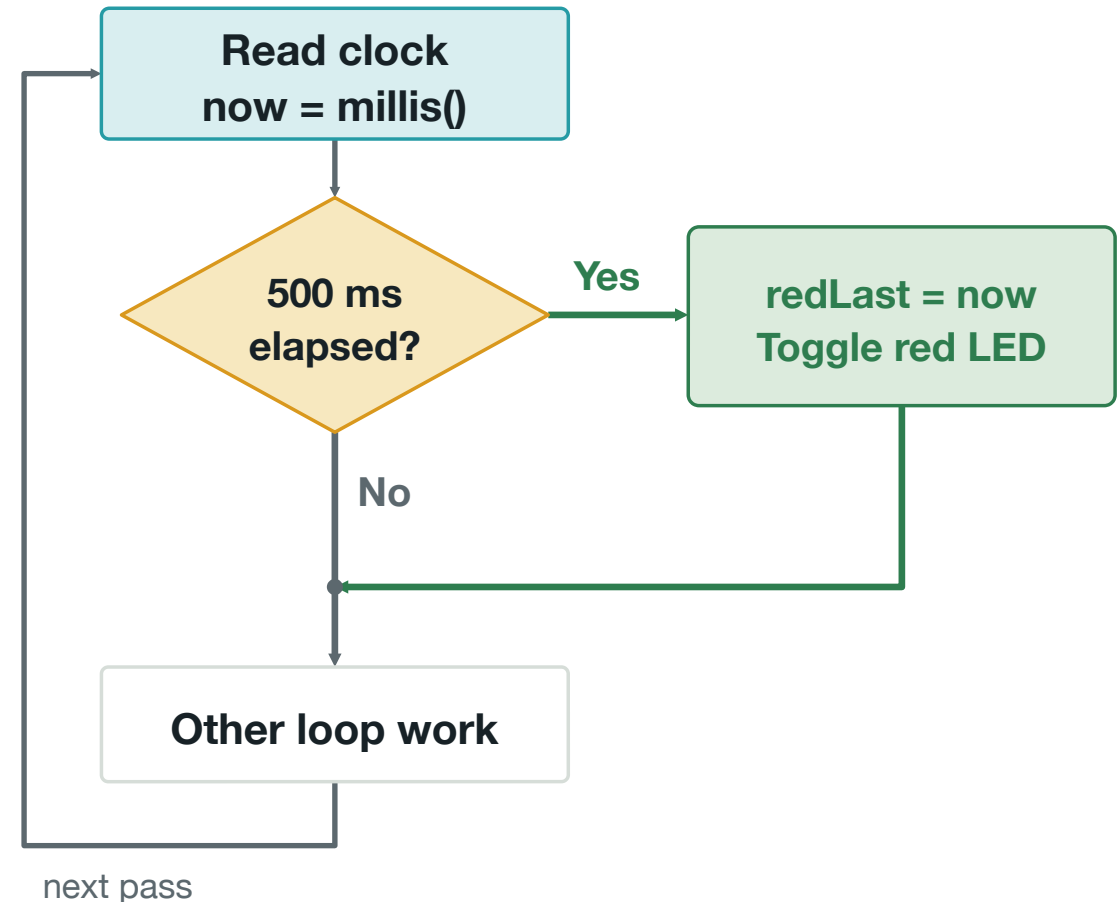
Clock-based timing in code

```
const int RED_LED = 3; // D2
uint32_t redLast = 0;
bool redOn = false;

void setup() {
  pinMode(RED_LED, OUTPUT);
  digitalWrite(RED_LED, LOW);
  redLast = millis();
}

void loop() {
  uint32_t now = millis();
  if (now - redLast >= 500) {
    redLast = now;
    redOn = !redOn;
    digitalWrite(RED_LED, redOn);
  }
  // Other loop work continues.
}
```

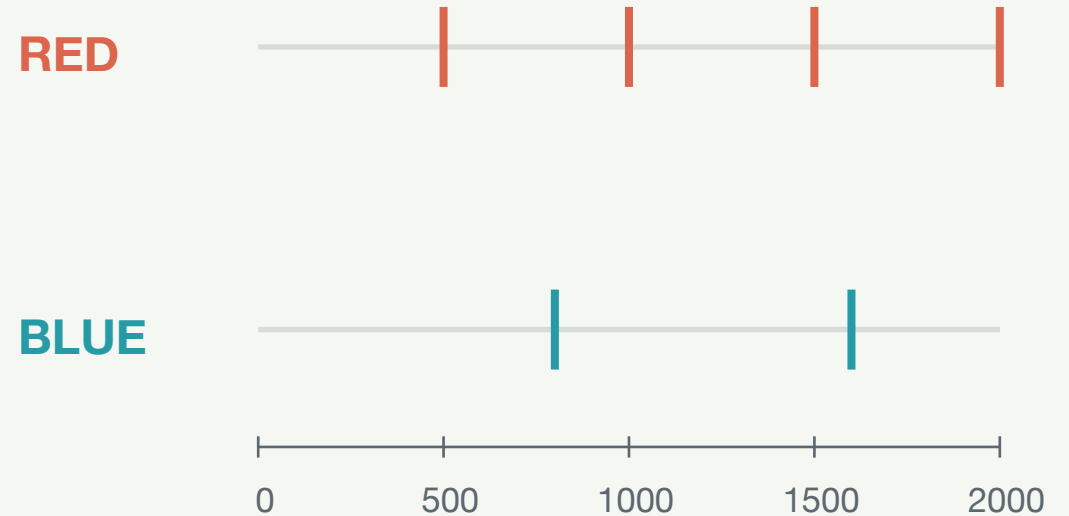
Nonblocking clock check



Independent timestamps for each output

Each output keeps its own state and timestamp.

```
void loop() {  
  uint32_t now = millis();  
  if (now - redLast >= 500) {  
    redLast = now;  
    red0n = !red0n;  
    digitalWrite(RED_LED, red0n);  
  }  
  if (now - blueLast >= 800) {  
    blueLast = now;  
    blue0n = !blue0n;  
    digitalWrite(BLUE_LED, blue0n);  
  }  
}
```



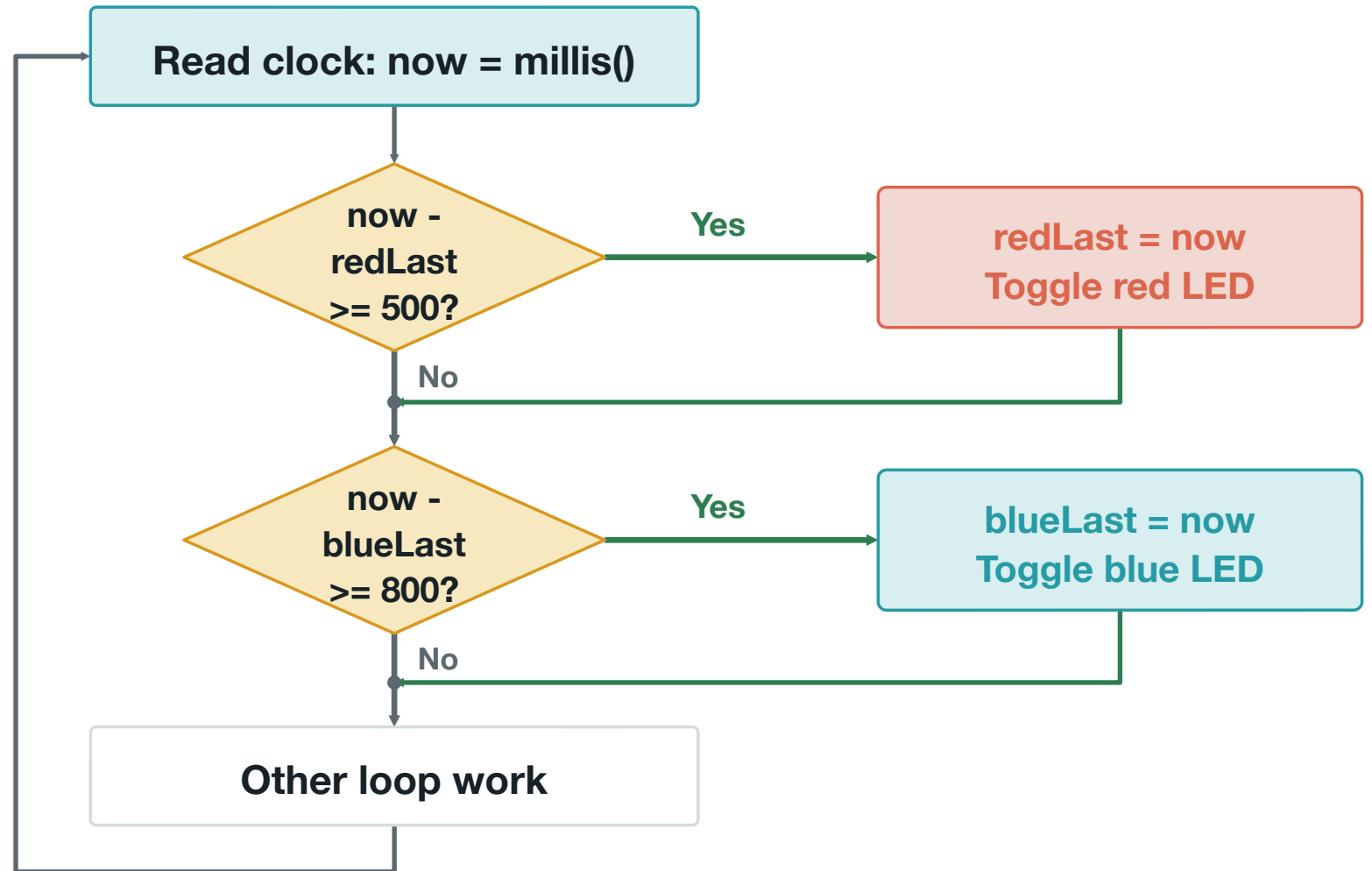
**Two separate if statements.
Both may run in one pass.**

Two checks in the same loop

In setup():
Both LEDs start LOW.
Save the initial value of
each timestamp.

A Yes branch rejoins
before the next check.

Saving a timestamp
does not reset millis().



A reusable periodic check

```
bool serviceDue(uint32_t nowMs,
                uint32_t intervalMs,
                uint32_t *lastRunMs) {
    uint32_t elapsedMs = nowMs - *lastRunMs;
    if (elapsedMs < intervalMs) {
        return false;
    }
    *lastRunMs = nowMs;
    return true;
}
```

False: skip this service.

True: update its timestamp and run it.

```
uint32_t lastSensorMs = 0;

void loop() {
    uint32_t nowMs = millis();
    if (serviceDue(nowMs, 50,
                   &lastSensorMs)) {
        int light = analogRead(A0);
        // Use the reading here.
    }
}
```

&lastSensorMs gives the helper access to the caller's persistent timestamp.